DAY 20

Testing

unittest · pytest · Fixtures · Mocking · Coverage · TDD

2 hrs	5 Concept Files	23 Practice Problems	Advanced Phase

30-DAY PYTHON PROGRAMMING COURSE
Phase 4: Advanced (Days 17-25)

SESSION OVERVIEW

Goal

By the end of this 2-hour session, students will be able to write automated tests for Python code using both **unittest** and **pytest**, use pytest fixtures for reusable setup/teardown, mock external dependencies with **unittest.mock**, read a code coverage report, and describe the Red-Green-Refactor Test-Driven Development (TDD) cycle.

Outcome

Students leave with 5 runnable concept files and a 23-problem practice set covering unittest basics, pytest fundamentals, fixtures, mocking, coverage, and TDD -- directly setting up Day 29 of the capstone project, where they will write a full pytest suite for the Task Manager API with 80%+ coverage.

Session Timeline

Time	Segment	Format
0:00 - 0:10	Why we test: manual vs automated checking	Discussion + demo
0:10 - 0:30	unittest.TestCase, assertions, setUp/tearDown	Live coding
0:30 - 0:50	pytest basics: plain asserts, parametrize	Live coding
0:50 - 1:00	Break	-
1:00 - 1:20	pytest fixtures: scopes, yield, chaining	Live coding
1:20 - 1:40	Mocking with unittest.mock: patch(), Mock, spec=	Live coding
1:40 - 1:55	Coverage reports + TDD walkthrough	Live coding
1:55 - 2:00	Wrap-up, practice problems assigned	Discussion

TIP

Testing is the single most requested skill gap reported by hiring managers for junior Python developers in India. Frame this session as a direct resume/interview asset, not just a course requirement.

Instructor Note: This is a concept-dense day. If running short on time, the Mocking section (1:20-1:40) is the safest to trim -- it can be reinforced again naturally during Day 23 (Web Requests & APIs) and Day 29 (capstone testing).

PREREQUISITES & PREPARATION

Requirements

Requirement	Details
Python version	3.12 (installed Day 1)
Packages needed	pytest, pytest-cov (unittest and unittest.mock are built-in)
Install command	pip install pytest pytest-cov --break-system-packages
Prior knowledge	Days 1-19: functions, OOP, error handling, dataclasses, type hints
Editor	VS Code with the Python extension (Day 1 setup)

Instructor Pre-Session Checklist

```
[ ] Confirm pytest and pytest-cov are installed in the demo environment
[ ] Pre-open day20_01 through day20_05 concept files in VS Code tabs
[ ] Have a terminal ready, already cd'd into the day's working folder
[ ] Test that `pytest -v` and `pytest --cov` both run without warnings
[ ] Review the Razorpay/Swiggy/BigBasket example contexts used throughout
[ ] Prepare a simple 'broken' function to demo a FAILING test live
```

STEP-BY-STEP INSTRUCTION GUIDE

Step 1: Why We Test & unittest Basics

"Today we're going to stop manually re-checking our code every time we change something. Imagine Priya is building a discount calculator for a food delivery app -- every change means she should be confident nothing else broke. That confidence is what automated testing gives us."

```
import unittest

def apply_discount(price, discount_percent):
    if discount_percent < 0 or discount_percent > 100:
        raise ValueError("invalid discount")
    return round(price - (price * discount_percent / 100), 2)

class TestApplyDiscount(unittest.TestCase):
    def test_ten_percent_discount(self):
        result = apply_discount(500, 10)
        self.assertEqual(result, 450.0)

# $ python -m unittest day20_01_unittest_basics.py -v
# test_ten_percent_discount ... ok
```

Error	Cause	Fix
AssertionError	Actual output != expected	Check your logic or your expected value
AttributeError: 'TestCase' has no attribute	Typo'd assertion method name	Check spelling: assertEqual, not assertEquals
No tests ran	Test method doesn't start with test_	Rename method to start with test_

DONE WHEN

Running `python day20_01_unittest_basics.py` prints 'OK' with the total test count and zero failures/errors.

Step 2: pytest Fundamentals

"unittest works, but notice how much boilerplate it needs -- a class, self, assertEqual. pytest lets us write a plain function with a plain assert ""statement, and still get rich failure output."

```
import pytest

def calculate_gst(amount, gst_percent=18.0):
    if amount < 0:
        raise ValueError("amount cannot be negative")
    return round(amount * gst_percent / 100, 2)

def test_gst_at_default_rate():
    assert calculate_gst(1000) == 180.0

@pytest.mark.parametrize("amount,pct,expected", [
    (1000, 18, 180.0), (500, 5, 25.0), (0, 18, 0.0),
])
def test_gst_parametrized(amount, pct, expected):
    assert calculate_gst(amount, pct) == expected

# $ pytest day20_02_pytest_basics.py -v
# 3 passed in 0.02s
```

OS	Run pytest with verbose output
macOS / Linux	pytest day20_02_pytest_basics.py -v
Windows (PowerShell)	pytest day20_02_pytest_basics.py -v
Windows (cmd.exe)	pytest day20_02_pytest_basics.py -v

TIP

pytest's command is identical across all three operating systems -- one less thing to troubleshoot compared to Day 1's environment setup!

Step 3: pytest Fixtures

"Every test in this section needs a fresh BankAccount. Instead of repeating setup code in every function, we write ONE fixture and pytest injects it ""automatically wherever a test asks for it by parameter name."

```
@pytest.fixture
def fresh_account():
    return BankAccount(owner="Meera Iyer", balance=1000.0)

def test_deposit_increases_balance(fresh_account):
    fresh_account.deposit(500)
    assert fresh_account.balance == 1500.0

@pytest.fixture(scope="module")
def mock_database_connection():
    conn = {"status": "connected"}
    yield conn   # <- setup above, teardown below
    print("disconnecting...")

# $ pytest day20_03_fixtures.py -v
# 13 passed in 0.02s
```

Error	Cause	Fix
fixture 'x' not found	Fixture name typo, or fixture defined in a different file without conftest.py	Match the parameter name exactly to the @pytest.fixt
Fixture runs every test unexpectedly	Default scope is 'function'	Use scope='module' or scope='session' for shared, ex

DONE WHEN

Each test using `fresh_account` gets its OWN independent BankAccount -- changing the balance in one test never leaks into another test.

Step 4: Mocking External Dependencies

"Arjun's function calls the real Razorpay payment API. We do NOT want our test suite hitting real servers every run -- it would be slow, cost money, and could even charge a real card! A Mock stands in for the real thing."

```
from unittest.mock import Mock, patch

def test_order_service_uses_repository():
    mock_repo = Mock(spec=OrderRepository)
    mock_repo.save.return_value = 42

    service = OrderService(repository=mock_repo)
    result = service.place_order("Priya Sharma", 1250.0)

    assert result["order_id"] == 42
    mock_repo.save.assert_called_once()

# $ pytest day20_04_mocking.py -v
# 12 passed in 0.02s
```

Error	Cause	Fix
AttributeError on mock call	Used spec=RealClass and called a method that doesn't exist on the real class	Check the real class's method names -- this is spec=
Mock still calls the real function	patch() target string points to the wrong module path	patch() the name as looked up in the CALLING modu

DONE WHEN

mock_repo.save.assert_called_once() passes with NO error output -- silence means the assertion succeeded.

Step 5: Coverage Reports & TDD Walkthrough

"Let's flip the usual order. Instead of writing code then tests, we write a FAILING test first, write the simplest code to pass it, then clean up. This is Red, Green, Refactor -- and it never lets your test suite fall behind."

```
# ROUND 1 - RED: write a failing test first
def test_no_late_fee_when_on_time():
    assert calculate_late_fee(0) == 0.0   # NameError: not defined yet

# ROUND 1 - GREEN: simplest code that passes
def calculate_late_fee(days_late):
    return 0.0

# ROUND 3 - final, after more RED/GREEN cycles:
def calculate_late_fee(days_late):
    if days_late <= 0:
        return 0.0
    if days_late <= 7:
        return days_late * 5.0
    extra = days_late - 7
    return (7 * 5.0) + (extra * 10.0)

# $ pytest --cov=day20_05_coverage_tdd --cov-report=term-missing
# Name                       Stmts  Miss  Cover
# day20_05_coverage_tdd.py     61     1    98%
```

WATCH OUT

The coverage report's 'Missing' column lists exact line numbers your tests never executed -- treat it as a checklist of tests still to write, not a grade to chase to 100% at all costs.

SESSION WRAP-UP

Student Checklist

```
[ ] I can write a unittest.TestCase with at least 3 assertion types
[ ] I can write a pytest test as a plain function with plain assert
[ ] I can use @pytest.mark.parametrize to cover multiple inputs in one test
[ ] I can write a pytest fixture and use it in a test function
[ ] I can explain the difference between fixture scopes (function vs module)
[ ] I can create a Mock object and set its return_value or side_effect
[ ] I can use patch() to replace a real dependency during a test
[ ] I can read a coverage report and identify untested lines
[ ] I can describe the Red-Green-Refactor TDD cycle in my own words
```

Key Concepts Recap

Concept	One-Line Summary
unittest.TestCase	Class-based tests; assertEqual/assertRaises/etc.
pytest function tests	Plain functions, plain assert, richer failure diffs
parametrize	One test definition, many input/expected pairs
Fixture	Reusable setup (+ teardown via yield), injected by name
Fixture scope	How often a fixture re-runs: function/class/module/session
Mock / MagicMock	Fake stand-in object you control and inspect
patch()	Temporarily replaces a real object with a Mock for one test
Coverage	% of source lines actually executed by the test suite
TDD	Red (failing test) -> Green (make it pass) -> Refactor (clean up)

Next Day Preview

Day 21 - Data & NumPy: Students move into the Data phase, learning NumPy arrays, broadcasting rules, vectorized operations, and basic linear algebra -- the foundation for Day 22's Pandas work and the capstone's analytics layer.

Troubleshooting Notes

If a student's pytest run silently collects 0 tests, the most common cause is running `pytest` from the wrong folder, or a filename that doesn't start with the pattern pytest auto-discovers -- remind them to pass the filename explicitly, exactly as shown in every 'How to Run' docstring this course uses.

QUICK REFERENCE CARD

Daily Workflow Commands

```
# Run a single file's tests with unittest
python -m unittest day20_01_unittest_basics.py -v

# Run a single file's tests with pytest
pytest day20_02_pytest_basics.py -v

# Run ALL test files in the current folder
pytest -v

# Run tests and show a coverage report
pytest --cov=my_module --cov-report=term-missing

# Run only tests whose name contains 'mock'
pytest -k mock -v
```

VS Code Shortcuts (Testing)

Shortcut	Action
Testing icon (flask) in sidebar	Open the Test Explorer panel
Click green play arrow next to a test	Run that single test
Ctrl+Shift+P -> 'Python: Configure Tests'	Set up pytest/unittest discovery
Ctrl+`	Open integrated terminal to run pytest manually

Tool Summary

Tool	Purpose
unittest	Built-in test framework; class-based, self.assertX methods
pytest	Third-party framework; plain functions, plain assert, fixtures
pytest-cov	Adds --cov coverage reporting to pytest
unittest.mock	Built-in mocking: Mock, MagicMock, patch()
monkeypatch	Built-in pytest fixture for lightweight patching (env vars, attrs)